# 4×4 Matrix Feedback Generator

disting NT plugin — system overview

A disting NT plugin that renders a self-oscillating 4-processor feedback network offline (throttled, non-real-time) into short audio buffers ("kits"), which a real-time sampler engine then plays back as one-shots or loops. There is no real-time connection between the plugin's input and output — rendering and playback are decoupled by a double-buffered kit store.

## 1. Signal network

Four channels, two 4×4 matrix mixers: an injection mixer feeding external sources in, and a feedback mixer routing every processor output to every processor input.

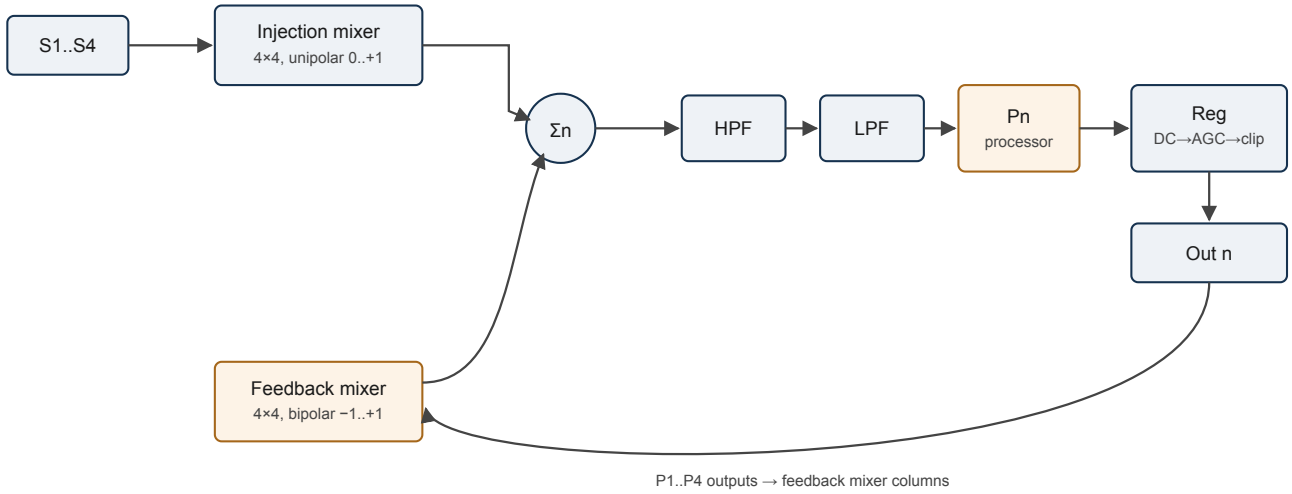



*Fig. 1 — Per-channel signal flow. Repeats for n = 1..4; the feedback mixer's 16 crosspoints connect every Pn output to every Σn input, covering all 24 elementary loop types (4 self, 6 two-processor, 8 three-processor, 6 four-processor).*

- **Injection mixer** (unipolar): four sources S1–S4 mixed independently into each channel. No inversion needed — sources aren't part of a loop.
- **Feedback mixer** (bipolar attenuverters): every processor output routed to every processor input.
- **Σn**: sums the injection mix and feedback mix for channel n.
- **HPF → LPF**: band-limits each channel before its processor; prevents loops from building energy outside the useful range. Ideally CV-controllable.
- **Pn**: four strongly nonlinear processors (frequency- and amplitude-dependent). One of the four is the frequency-domain pitch shifter (§2).
- **Reg** (regulator): DC block → AGC (drives loop gain toward 1, attack ~2–10 ms, release ~100–500 ms) → soft clip (tanh, never hard clip). Sits before the output split, so every loop and every output tap is protected.
- **Out n**: regulated output, split to the audio output and back into feedback-mixer column n.

## 2. Pitch-shift processor

One of the four processors implements frequency-domain pitch shifting, based on Ocean's algorithm (Juillerat, 2010) — the same technique as the reference implementation this design started from.

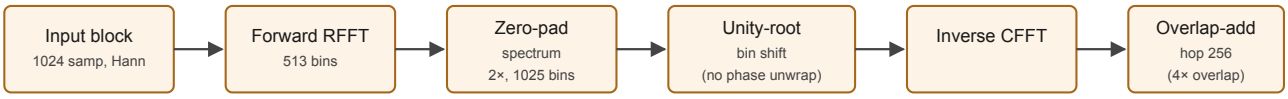



*Fig. 2 — Pitch-shift processor pipeline. All blocks are hand-written float C++ (no FFT/DSP library is exposed by the disting NT API) with statically-sized buffers.*

The reference implementation used TI C28x hardware-accelerated FFT calls (`RFFT_f32`/`ICFFT_f32`). The disting NT API exposes **no FFT or DSP library** (verified against `api.h`, `wav.h`, `slot.h`, `serialisation.h`, `microtuning.h`) — the forward/inverse FFT must be hand-written in portable float C++ using only `<cmath>`, with statically-sized buffers.

This processor's block latency (at least one hop, realistically several for overlap-add to settle — several ms) becomes a structural property of whichever feedback loop it sits in, shaping that leg's timing/character rather than being a hidden implementation detail. This only affects the internal texture of the render — it has no bearing on playback, since playback never has to track the render's timeline in real time.

## 3. Execution model: render vs. playback

The plugin's `step()` does two jobs of very different weight.

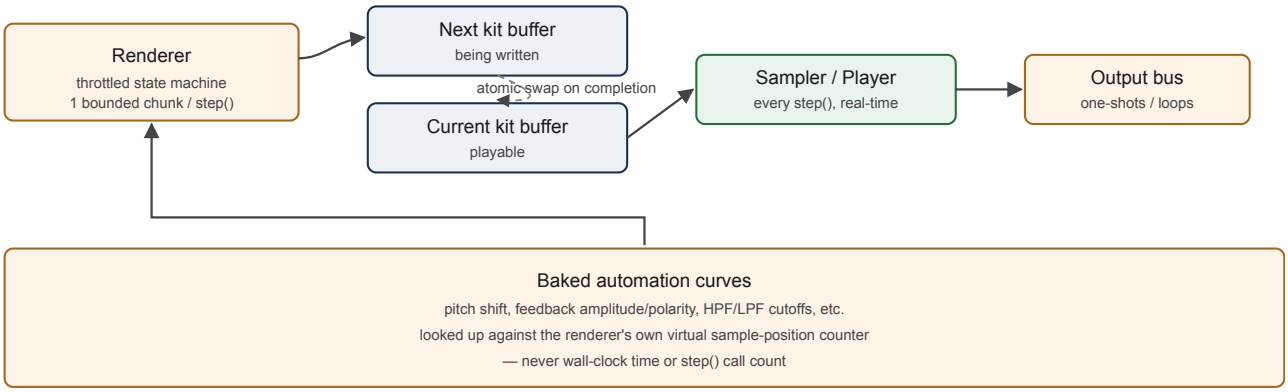



*Fig. 3 — Render/playback decoupling via double-buffered kits. The renderer only ever writes the inactive buffer; the player only ever reads the active one.*

1. **Player (always-on, real-time):** reads from the *current kit* buffer and plays slices as one-shots or loops. Comparable in cost to a basic sample-playback algorithm — this is the plugin's entire real-time obligation.
2. **Renderer (background, throttled):** advances the feedback-network computation (matrices, filters, processors, regulators) by one small bounded chunk per `step()` call — an explicit state machine, since the platform has no threads and `step()` must never block. Produces a fixed-length buffer (e.g. 5 seconds) at whatever pace the chunk size allows (render speed is a free design parameter — e.g. ~4× slower than real-time is one possible operating point, not a requirement).

**Double buffering:** a *current kit* (playable) and a *next kit* (being rendered). The renderer only ever writes into the inactive buffer; the pointer swap to make it the new *current kit* happens atomically, only once the render is fully complete. The player never sees a partially-rendered buffer.

Baked parameter-automation curves (pitch shift amount, feedback amplitude/polarity, HPF/LPF cutoffs, and others) are evaluated against the **renderer's own internal virtual sample-position counter**, not wall-clock time or `step()` call count — this keeps automation correct regardless of how fast or slow the render chunking happens to run.

## 4. CPU budget strategy

Because the renderer has no deadline (the player just keeps using whichever kit is current until a new one swaps in), render CPU cost is a tunable design parameter rather than an emergent property of the algorithm:

- **Always-on cost:** player only — small, constant, every `step()` call.
- **Render cost:** incurred only while actively baking a new kit; chunked into small bounded units (e.g. one FFT butterfly stage, or one stage of the matrix/filter/regulator pass) sized so that render-chunk cost + player cost + headroom for other algorithms in the same preset stays well under the module's CPU-overload threshold on *every single call* — average cost over a render cycle is not the relevant number, peak per-call cost is.
- Smaller chunks → lower peak CPU, longer bake time. Larger chunks → faster bake, higher (but still bounded and chosen) peak cost. This trade is free to make because nothing downstream has a deadline.

> Reference point: the LASER example plugin runs 20–30% CPU as a genuine real-time algorithm with a forced per-call cost. This design's forced per-call cost (the player) should be much lower, since the expensive part is optional and schedulable rather than forced every call — the actual number can only come from on-device profiling.

## 5. Memory

All allocation is static, sized via `calculateRequirements()` — no `new`/`malloc` at runtime anywhere, per the platform's hard constraint.

| Item | Approx. size | Notes |
|---|---|---|
| Kit buffer (5s mono, 48kHz float) | ~960 KB | ×2 for double buffering (~1.9 MB+) — needs verification against actual available DRAM, not confirmed in project docs |
| FFT/pitch-shifter scratch (1024/2048-sample blocks, spectra, overlap-add tail) | a few KB | |
| Matrix coefficients, filter/regulator state | negligible | |
| Baked automation curve tables | unknown | depends on curve length/resolution — not yet specified |

## 6. Open design questions

- **State continuity between renders:** does the feedback network's internal state (filters, AGC envelopes, matrix history) carry over from the end of one render into the start of the next (continuous evolution), or does each render start cold with a fresh source injection (per the "kick-start loops" playing note)? Affects both sound character and per-render compute (settling time).
- **Render chunk granularity and resulting bake time:** only determinable by on-device profiling.
- **Actual available DRAM budget** for double-buffered kit storage: unverified.
- **Sampler slice/trigger design:** fixed-size slices vs. transient-detected, one-shot vs. loop selection, polyphony — not yet specified.
- **AGC vs. pitch-shifter latency interaction:** the AGC's fast attack (~2–10 ms) operates in a similar time range to the pitch-shift leg's inherent block latency; possible periodic pumping tied to hop boundaries needs listening/tuning, not just calculation.

## 7. Platform constraints honored

- No dynamic allocation — placement `new` on firmware-provided static memory only.
- No exceptions, no RTTI, no STL, no threads, no OS/file calls inside `step()`.
- FFT implemented from scratch in portable float C++ (`<cmath>` only) since the API provides none.
- `step()` never blocks — render work is chunked as an explicit state machine, never a single long pass.